

Online Stock Trading Platform (Paper Trading) — Implementation PRD

Java 17 · Jakarta Servlet 6 / JSP / JSTL · JDBC · MySQL 8 · Tomcat 10.1 · Maven Team: 4 · Time: 3 days · Audience: college evaluators

`schema.sql` and `seed.sql` shipped alongside this file were executed against a real MySQL 8 instance. The CHECK constraints, the P&L invariant and the rollback behaviour described here were verified in SQL. The Java code was not compiled, because it does not exist yet. That is your job.

1. Executive Summary

A simulated stock trading web app. Users register, get ₹100,000 of virtual cash, browse 15 seeded stocks, buy and sell at the current mock price, and see holdings and profit/loss. An admin manages users and stocks, edits prices, and watches trades. No real money, brokers, or market data.

What earns marks here is not the UI. It is:

- A JDBC transaction around every trade (lock, validate, update cash, update holdings, insert trade, commit or roll back).
- A clean Servlet → Service → DAO layering.
- A background thread that moves prices.
- Custom exceptions that map to user-visible errors.
- Role-based access through a Filter and sessions.

Everything else is decoration, and decoration is the first thing to cut when you are behind.

Hard decisions already made for you (details in later sections):

Decision	Choice	Reason
Framework	Plain Servlets + JSP + JSTL. No Spring.	Spring hides Servlets and JDBC, which are the graded topics. It also costs a day of learning.
Tables	4 (<code>users</code> , <code>stocks</code> , <code>holdings</code> , <code>trades</code>)	An <code>orders</code> table is useless when every order executes instantly.
Order type	Market order, instant execution	Limit/stop orders need a matching engine. Cut.
Stock data	Seeded in SQL + admin edits + one optional background price thread	Works offline. Deterministic for the demo.
Password hashing	PBKDF2 from the JDK (<code>javax.crypto</code>), no library	Zero dependencies.
Money type	<code>BigDecimal</code> in Java, <code>DECIMAL</code> in MySQL. Never <code>double</code> .	A graded-for-free correctness point.
Connection pool	None. <code>DriverManager</code> , one connection per request.	A pool is one more thing to configure. Not needed for a 5-user demo.
Admin login	Same login page, redirect by role	A second login page is padding.

2. Problem Statement

Students learning stock markets have nowhere to practise without risking money. Real broker platforms need KYC and funds, and their APIs are rate-limited and unreliable for demos. This project provides a controlled simulation: virtual cash, mock prices, and the same buy/sell/portfolio mechanics, so trading logic can be exercised safely.

Academic angle: the same problem is a clean showcase of transactional integrity. Buying a stock changes cash, holdings and the trade ledger together. Either all three change or none do.

3. Project Objectives

1. Implement a working multi-user web app using Servlets, JSP, JDBC and MySQL.
2. Implement buy/sell as ACID transactions with explicit commit and rollback, including row locking for concurrent requests.
3. Demonstrate OOP with real uses: inheritance and polymorphism in the order-execution code, interfaces in the price simulator, encapsulation in the models.
4. Demonstrate collections, custom exceptions and multithreading where they naturally occur.
5. Provide role-based access (USER / ADMIN) using sessions and a servlet Filter.
6. Be demonstrable in 8–10 minutes with no internet.

4. Feasibility Verdict

Audit of the scope as originally described

Factor	Score /10	Notes
3-day deadline	5	11 user screens + 6 admin screens + trading + P&L + simulator + docs + testing is tight.
4-person team	7	Parallelisable if contracts are frozen early.
Java complexity	7	Servlets/JDBC are fine. Transactions and locking are where students stumble.
Database complexity	8	4 tables. Easy.
Frontend complexity	6	JSP + JSTL pitfalls (Section 20 and 29) will eat hours. No JS frameworks.
Integration risk	5	Four people editing one webapp. Merge conflicts and "works on my machine" Tomcat/MySQL mismatches.
Demo reliability	8	Offline-capable. Only danger is the price thread changing numbers mid-demo (handled: default OFF).
Overall (original scope)	6.0 / 10	Doable but with no slack. One sick teammate sinks it.

After the cuts in this PRD

Factor	Score /10
3-day deadline	7
4-person team	8 (file ownership defined, no shared files)
Java complexity	8
Database complexity	9
Frontend complexity	7
Integration risk	7
Demo reliability	9
Overall	7.8 / 10

Verdict: **GO**, with the scope in this document. The remaining risk is mostly environment setup (Tomcat 9 vs 10, JSTL URI errors, ₹ rendering as ???) and Day-3 integration. Both have explicit mitigations in Section 29.

What was cut in the audit to get from 6.0 to 7.8: the `orders` table, a separate admin login page, buy/sell confirmation pages, user deletion, admin balance reset, pagination, charts, price history, watchlist, forgot-password, email verification, connection pooling, CSRF tokens. All listed in Section 31.

5. Target Users & Roles

Role	Can do	Cannot do
Guest	Register, log in	Everything else
USER	Browse market, trade, view portfolio/history, edit profile	Access <code>/admin/*</code>
ADMIN	Manage users (block/unblock), manage stocks, edit prices, view all trades and stats	Trade. Admin accounts are not traders; <code>/app/*</code> redirects them to the admin dashboard.

Two separate realms (`/app/*` for users, `/admin/*` for admins) keeps the authorization Filter to about 25 lines.

6. Scope

In scope: everything in the MUST HAVE list below, plus SHOULD HAVE only if the Day-2 checkpoint is green.

Out of scope: real money, brokers, payments, KYC, live market data, anything in Section 31.

The UI shows a permanent banner: "Paper trading simulator. Virtual money. Mock prices. Not real market data." Do not skip this. Seeded prices are invented and must never be presented as real quotes.

7. MUST HAVE / SHOULD HAVE / CUT

MUST HAVE (P0)

1. Register (auto-credit ₹100,000), login, logout, session timeout, role-based access
2. View profile, edit name/email, change password
3. Market list with search by symbol/company
4. Stock detail page with Buy and Sell forms
5. Buy and sell with a JDBC transaction, full validation, and the three custom exceptions for insufficient cash/holdings/invalid order
6. Portfolio: quantity, avg buy price, current price, invested, current value, unrealized P&L; cash; net worth
7. Trade history (last 100)
8. Dashboard: cash, net worth, total P&L, last 5 trades
9. Admin: user list + block/unblock; stock add/edit/deactivate; manual price update; all-trades view; stats (counts, total traded value, top stocks by trade count)
10. Seed data (3 users, 15 stocks, 4 trades)
11. Rollback demo toggle (fault injection, Section 11)

SHOULD HAVE (P1)

1. Realized P&L shown on portfolio/dashboard (the column exists already; showing it is a single query)
2. Background price simulator thread with admin on/off switch
3. `ConcurrentTradeDemo` standalone class: 10 threads, one user, proves cash never goes negative
4. JUnit tests for `PnLCalculator` and `PasswordUtil`
5. Sort market table by symbol / price / change %
6. Day change (price vs `prev_price`) with green/red colouring

CUT

See Section 31. If you are tempted to add something not in MUST or SHOULD, the answer is no.

Cut rule: at the Day-2 evening checkpoint, if buy/sell + portfolio do not work end-to-end, every P1 item is cancelled immediately and everyone helps with P0 bugs.

8. Functional Requirements

ID	Requirement	Pri
FR-01	Register with username, email, full name, password. Username 3-30 chars [A-Za-z0-9_]; email valid format; password ≥ 8 chars with at least one letter and one digit.	P0
FR-02	Username and email must be unique; show a specific error.	P0
FR-03	New USER gets ₹100,000.00 cash (DB default).	P0
FR-04	Login with username + password. Blocked users cannot log in. Error message is identical for unknown user and wrong password.	P0
FR-05	Session timeout 15 minutes; logout invalidates the session.	P0
FR-06	All <code>/app/*</code> pages require login as USER; all <code>/admin/*</code> pages require ADMIN.	P0
FR-07	Market page lists active stocks: symbol, name, sector, price, change %. Search by text.	P0
FR-08	Stock detail shows info, price, user's current holding, and Buy/Sell forms.	P0
FR-09	Buy: integer quantity 1-10,000; cost = price $\times$ quantity; reject if cost > cash; stock must be active.	P0
FR-10	Sell: integer quantity 1-10,000; reject if quantity > held. Selling is allowed even for deactivated stocks.	P0
FR-11	Every buy/sell is a single DB transaction. Failure anywhere leaves cash, holdings and trades untouched.	P0
FR-12	Execution price = the stock price read inside the transaction, not the price the browser displayed. The UI says "indicative price".	P0
FR-13	Portfolio shows per-holding and total P&L (Section 12).	P0
FR-14	History shows the user's last 100 trades, newest first.	P0
FR-15	Profile: edit name and email; change password (needs old password).	P0
FR-16	Admin can block/unblock any USER (not themselves, not other admins). A blocked user's trade attempts are rejected even if the session is still alive.	P0
FR-17	Admin can add a stock, edit name/sector/description, deactivate/reactivate. No hard delete (foreign keys).	P0
FR-18	Admin can set a stock's price manually (must be > 0).	P0
FR-19	Admin sees the last 200 trades across all users.	P0
FR-20	Admin dashboard: total users, active stocks, total trades, total traded value, top 5 stocks by trade count.	P0
FR-21	Background price simulator (random walk), admin on/off switch.	P1
FR-22	Demo fault-injection switch (forces a failure mid-trade to show rollback).	P0

Non-functional (kept to what you can actually satisfy): UTF-8 end to end (₹ must render); all SQL via `PreparedStatement`; all money as `BigDecimal`; all output HTML-escaped via `<c:out>`; page response < 1 s on a laptop with the seed data; runs with no internet.

9. User Workflows

Register → first trade 1. `/register` → fill form → POST. On validation error, redisplay form with messages and the entered values (except password). 2. On success: redirect to `/login` with flash "Account created. Log in." 3. Login → session created → redirect to `/app/dashboard`. 4. Market → click a stock → detail page → enter quantity → Buy. 5. Success → redirect to `/app/portfolio` with flash "Bought 10 TCS @ ₹3,900.00". Failure → back to stock page with red error.

Always use **Post-Redirect-Get** after a successful trade. Otherwise F5 re-submits the order and the user buys twice. Evaluators love to hit refresh.

Sell: same, from the stock detail page or a "Sell" link in the portfolio table that goes to the stock detail page.

10. Admin Workflows

1. Login with admin account → `/admin/dashboard` (stats).
2. Users → table → Block/Unblock button (POST).
3. Stocks → table + add form; click Edit to populate the form (`?edit=<id>`); Deactivate/Activate button.
4. Prices → per-stock inline price field + Update; simulator on/off; fault-injection toggle (labelled "DEMO TOOL").
5. Trades → read-only table of the last 200 trades.

11. Trading Logic

Rules

- Market orders only, executed instantly at `stocks.price`.
- No brokerage, no taxes, no margin, no short selling, no partial fills.
- Quantity is an `int`. Max 10,000 per order.
- Users cannot trade while `status = BLOCKED`.

The transaction (exact JDBC flow)

Design rules: - **DAOs never open or close connections and never commit**. They take a `Connection` as the first parameter. The **Service** owns the connection and the transaction boundary. That is the whole point of the layering and it is what you explain to the examiner. - `autoCommit(false)` at the start, `commit()` once at the end, `rollback()` in every failure path. - Lock order is always: **user row → holding row**. Fixed lock order prevents deadlocks.

```
// TradingService.java (shared template; executor = Buy or Sell strategy)
private Trade placeOrder(OrderExecutor executor, long userId, long stockId, int qty)
    throws ApplicationException {
    validateQuantity(qty); // 1..10000, else InvalidOrderException
    try (Connection c = DBConnection.getConnection()) {
        try {
            c.setAutoCommit(false);
            Trade trade = executor.execute(c, userId, stockId, qty);
            c.commit();
            return trade;
        } catch (AppException | RuntimeException e) { // business failure or bug
            rollbackQuietly(c);
            throw e;
        } catch (SQLException e) { // DB failure
            rollbackQuietly(c);
            throw new DataAccessException("Trade failed; no changes were saved.", e);
        } finally {
            try { c.setAutoCommit(true); } catch (SQLException ignored) { }
        }
    }
}
```

`OrderExecutor.execute(...)` must declare `throws ApplicationException, SQLException`. If it does not, the compiler rejects the `catch (SQLException e)` block because the checked exception is never thrown.

BUY (`BuyOrderExecutor.execute`)

```
1. user = userDao.findByIdForUpdate(c, userId) -- SELECT ... FOR UPDATE (locks the row)
   null -> UserNotFoundException
   BLOCKED/ADMIN -> InvalidOrderException("Account cannot trade")
2. stock = stockDAO.findById(c, stockId) -- plain SELECT; read price HERE
   null -> StockNotFoundException
   !isActive -> InvalidOrderException("Stock not available for trading")
3. total = stock.price * qty -- BigDecimal, scale 2
   user.cash < total -> InsufficientBalanceException(available, required)
4. userDao.adjustCash(c, userId, total.negate()) -- UPDATE users SET cash_balance = cash_balance + ?
5. h = holdingDAO.findForUpdate(c, userId, stockId)
   none -> holdingDAO.insert(qty, price)
   exists -> newQty = h.qty + qty
               newAvg = (h.qty*h.avg + qty*price) / newQty (scale 4, HALF_UP)
               holdingDAO.update(c, userId, stockId, newQty, newAvg)
6. FaultInjector.checkpoint("after-holding-update") -- throws RuntimeException if demo toggle is on
```

```
7. tradeDAO.insert(c, BUY, qty, price, total, realizedPnl = 0)
-- caller commits
```

SELL (SellOrderExecutor.execute)

```
1. user = findByIdForUpdate (same checks as buy)
2. stock = stockDAO.findById (must exist; may be inactive)
3. h = holdingDAO.findForUpdate(c, userId, stockId)
   none or h.qty < qty -> InsufficientHoldingsException(held, requested)
4. total = price * qty
   realized = (price - h.avg) * qty, scale 2, HALF_UP
5. newQty = h.qty - qty
   newQty == 0 -> holdingDAO.delete(...) (CHECK quantity > 0 forbids a zero row)
   else -> holdingDAO.update(c, ..., newQty, h.avg) -- average does NOT change on a sell
6. userDAO.adjustCash(c, userId, total)
7. FaultInjector.checkpoint("after-holding-update")
8. tradeDAO.insert(c, SELL, qty, price, total, realized)
```

Why these specific details

- **SELECT ... FOR UPDATE on the user row first** serialises all orders from the same user. Two browser tabs or a double-click cannot both pass the "enough cash?" check. Without this, the check-then-update pattern is a textbook lost-update race.
- **cash_balance = cash_balance + ? (atomic SQL arithmetic)** rather than "read in Java, compute, write back". Same reason.
- **CHECK (cash_balance >= 0)** is the last line of defence. Even if you botch the Java logic, MySQL refuses to go negative and the whole transaction rolls back. Verified on MySQL 8: error 3819.
- **Price is read inside the transaction.** If the simulator moves the price between page render and submit, the order executes at the current price. That is what real markets do. Say it out loud in the demo.
- **ENGINE=InnoDB is mandatory.** MyISAM silently ignores transactions. MySQL 8 defaults to InnoDB, but the schema states it explicitly anyway.
- **The transaction template exists once,** in `TradingService`. Buy and sell differ only in the executor. That is a legitimate use of polymorphism (Section 22), not a decorative one.

Rollback demo (FR-22)

`FaultInjector` has a static volatile boolean `failMidTrade`. The admin Prices page toggles it. When on, `checkpoint()` throws `IllegalStateException("Injected failure")` after cash and holdings have already been updated but before the trade row is inserted. The service rolls back; the user sees "Trade failed; no changes were saved"; the examiner sees cash and holdings unchanged in MySQL Workbench. It is fault injection, a standard testing technique. Label it clearly as a demo tool and do not hide that it is simulated.

Concurrency demo (P1, ConcurrentTradeDemo)

Standalone main using `ExecutorService` with 10 threads, all calling `tradingService.buy(rahulId, RELIANCE, 15)`. RELIANCE is ₹2,850, so one order costs ₹42,750 and ₹100,000 funds only 2. Expected: exactly 2 succeed, 8 throw `InsufficientBalanceException`, final cash = ₹14,500.00. Run it with the simulator OFF so the price is stable. This is deterministic and impressive.

12. P&L Logic

Method: **average cost**. Simple, standard, implementable.

Quantity	Formula	Where computed
Invested value	<code>qty × avgBuyPrice</code>	<code>PnLCalculator</code>
Current value	<code>qty × currentPrice</code>	<code>PnLCalculator</code>
Unrealized P&L	<code>currentValue - invested</code>	<code>PnLCalculator</code>
Unrealized %	<code>unrealized / invested × 100</code> (guard invested = 0)	<code>PnLCalculator</code>
Realized P&L (per sell)	<code>(sellPrice - avgBuyPrice) × qty</code> , stored in <code>trades.realized_pnl</code> at sell time	<code>SellOrderExecutor</code>
Total realized	<code>SELECT COALESCE(SUM(realized_pnl),0) FROM trades WHERE user_id=?</code>	<code>TradeDAO.sumRealizedPnl</code>
Net worth	<code>cash + Σ currentValue</code>	<code>PortfolioService</code>
Overall P&L	<code>netWorth - 100,000</code>	<code>PortfolioService</code>

Invariant you can use as a test: overall P&L = realized + unrealized. Verified on the seed data for priya: cash ₹21,600 + holdings value ₹(10×3900 + 20×1500 + 30×430 = 81,900) = ₹103,500; minus 100,000 = ₹3,500 = ₹600 realized + ₹2,900 unrealized. Tolerance in tests: ±₹1.00 (average price is stored to 4 decimals, so rounding can drift by cents on large lots).

Tradeoff made: realized P&L is stored per sell trade instead of being recomputed from history. That costs one extra column and makes the feature trivial. FIFO lot accounting, tax lots and per-day P&L are cut. This invariant breaks if you let the admin reset a user's balance, which is why that feature is cut.

Rounding rule everywhere: `RoundingMode.HALF_UP`; money scale 2; average price scale 4. Never `new BigDecimal(double)`; use `BigDecimal.valueOf` or strings.

`PnLCalculator` should be a pure static class with no DB access, so it can be unit tested in 10 minutes.

13. Database Design

Four tables. Analysis of the "obvious" ones:

Candidate	Decision	Reason
users	Keep	Identity, role, status and cash balance (one column, because every user has exactly one cash account).
stocks	Keep	Master data plus current price. <code>prev_price</code> gives day-change colouring without a price-history table.
holdings	Keep	Current position per (user, stock). Composite PK (<code>user_id</code> , <code>stock_id</code>) enforces one row per pair; no surrogate key needed.
trades	Keep	Immutable ledger. Carries <code>realized_pnl</code> .
orders	Reject	Orders execute instantly, so an order is a trade. A second table only adds joins and a status machine.
portfolios	Reject	A portfolio is just a user's holdings.
price_history	Reject	Only needed for charts. Charts are cut.

Table summary

Table	Columns	PK	FKs	Constraints	Purpose
users	id, username, email, full_name, password_hash, role, status, cash_balance, created_at	id	-	UNIQUE(username), UNIQUE(email), CHECK cash ≥ 0, ENUM role/status	Accounts and cash
stocks	id, symbol, company_name, sector, description, price, prev_price, is_active, updated_at	id	-	UNIQUE(symbol), CHECK price > 0, CHECK prev_price > 0	Tradable instruments
holdings	user_id, stock_id, quantity, avg_buy_price	(user_id, stock_id)	user_id→users, stock_id→stocks (RESTRICT)	CHECK quantity > 0, CHECK avg > 0	Current positions
trades	id, user_id, stock_id, trade_type, quantity, price, total_amount, realized_pnl, executed_at	id	user_id→users, stock_id→stocks (RESTRICT)	CHECK qty > 0, CHECK price > 0, ENUM type, INDEX(user_id, executed_at)	Trade ledger

Design notes:

- `ON DELETE RESTRICT` is why users and stocks are never hard-deleted. Users: block. Stocks: deactivate.
- `password_hash` format: `Base64(salt):Base64(PBKDF2-SHA256 hash)`. 255 chars is plenty.
- MySQL 5.7 parses `CHECK` but **ignores** it. You need MySQL 8.0.16+. Verify with `SELECT VERSION()`; on every team machine on Day 1.
- Create the database with `utf8mb4` or ₹ becomes ?.

schema.sql

```
-- Paper Trading Platform: schema (MySQL 8.0.16+ required: CHECK constraints are enforced from this version)
DROP DATABASE IF EXISTS paperttrade;
CREATE DATABASE paperttrade CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE paperttrade;

CREATE TABLE users (
  id          BIGINT          NOT NULL AUTO_INCREMENT,
  username    VARCHAR(30)     NOT NULL,
  email       VARCHAR(100)    NOT NULL,
  full_name   VARCHAR(100)    NOT NULL,
  password_hash VARCHAR(255)  NOT NULL,
  role        ENUM('USER','ADMIN') NOT NULL DEFAULT 'USER',
  status      ENUM('ACTIVE','BLOCKED') NOT NULL DEFAULT 'ACTIVE',
  cash_balance DECIMAL(15,2) NOT NULL DEFAULT 100000.00,
  created_at  TIMESTAMP      NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  UNIQUE KEY uq_users_username (username),
  UNIQUE KEY uq_users_email (email),
  CONSTRAINT chk_users_cash CHECK (cash_balance >= 0)
) ENGINE=InnoDB;

CREATE TABLE stocks (
  id          BIGINT          NOT NULL AUTO_INCREMENT,
  symbol      VARCHAR(15)     NOT NULL,
  company_name VARCHAR(100)   NOT NULL,
  sector      VARCHAR(50)     NULL,
  description  VARCHAR(500)   NULL,
  price       DECIMAL(12,2)   NOT NULL,
  prev_price  DECIMAL(12,2)   NOT NULL,
  is_active   TINYINT(1)      NOT NULL DEFAULT 1,
  updated_at  TIMESTAMP      NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  UNIQUE KEY uq_stocks_symbol (symbol),
  CONSTRAINT chk_stocks_price CHECK (price > 0),
  CONSTRAINT chk_stocks_prev CHECK (prev_price > 0)
) ENGINE=InnoDB;

CREATE TABLE holdings (
  user_id     BIGINT          NOT NULL,
  stock_id    BIGINT          NOT NULL,
  quantity    INT             NOT NULL,
```

```

avg_buy_price DECIMAL(12,4) NOT NULL,
PRIMARY KEY (user_id, stock_id),
CONSTRAINT fk_holdings_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT,
CONSTRAINT fk_holdings_stock FOREIGN KEY (stock_id) REFERENCES stocks(id) ON DELETE RESTRICT,
CONSTRAINT chk_holdings_qty CHECK (quantity > 0),
CONSTRAINT chk_holdings_avg CHECK (avg_buy_price > 0)
) ENGINE=InnoDB;

CREATE TABLE trades (
  id BIGINT NOT NULL AUTO_INCREMENT,
  user_id BIGINT NOT NULL,
  stock_id BIGINT NOT NULL,
  trade_type ENUM('BUY','SELL') NOT NULL,
  quantity INT NOT NULL,
  price DECIMAL(12,2) NOT NULL,
  total_amount DECIMAL(15,2) NOT NULL,
  realized_pnl DECIMAL(15,2) NOT NULL DEFAULT 0.00,
  executed_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (id),
  KEY idx_trades_user_time (user_id, executed_at),
  KEY idx_trades_stock (stock_id),
  CONSTRAINT fk_trades_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE RESTRICT,
  CONSTRAINT fk_trades_stock FOREIGN KEY (stock_id) REFERENCES stocks(id) ON DELETE RESTRICT,
  CONSTRAINT chk_trades_qty CHECK (quantity > 0),
  CONSTRAINT chk_trades_price CHECK (price > 0)
) ENGINE=InnoDB;

```

seed.sql

Seed users: admin / Admin@123, rahul / Demo@123 (fresh, ₹100,000, used for the live demo), priya / Demo@123 (pre-populated: 3 holdings, 4 trades, ₹600 realized P&L, so Dev 4 has real data on Day 1 without waiting for the buy flow). The hashes below are real PBKDF2 output (65,536 iterations, 256-bit) generated with the same parameters PasswordUtil must use. Seeded prices are invented.

```

USE papertrade;

-- Passwords: admin -> Admin@123 ; rahul, priya -> Demo@123
-- Hash format: Base64(salt):Base64(PBKDF2WithHmacSHA256, 65536 iterations, 256-bit)
INSERT INTO users (id, username, email, full_name, password_hash, role, status, cash_balance) VALUES
(1,'admin','admin@papertrade.local','Platform Admin','0+3DUVGW/WjflstenwOF3w==:opvQZC0rEjgqCLs00nh8rXwe0BhILK0+zdI/c1ELK0k=', 'ADMIN', 'ACTIVE', 0.00),
(2,'rahul','rahul@papertrade.local','Rahul Sharma','GkhX+Tj0sacIqQ0GZC6XA==:Uqu0H3XXIMG9LS3wtbuIwdmk6fbIXMcDbHtsSaZji4E=', 'USER', 'ACTIVE', 100000.00),
(3,'priya','priya@papertrade.local','Priya Verma','wBU07JGZc/buhmqgpMrkaw==:t2pmqPYP34CL8gfaMhpqRZt89Vn7RuVsK7n0Nk4y0Wk=', 'USER', 'ACTIVE', 21600.00);

-- Mock prices. NOT real market quotes.
INSERT INTO stocks (id, symbol, company_name, sector, description, price, prev_price) VALUES
(1,'RELIANCE','Reliance Industries Ltd','Energy','Conglomerate: energy, retail, telecom.',2850.00,2850.00),
(2,'TCS','Tata Consultancy Services','IT','IT services and consulting.',3900.00,3900.00),
(3,'INFY','Infosys Ltd','IT','IT services and consulting.',1500.00,1500.00),
(4,'HDFCBANK','HDFC Bank Ltd','Banking','Private sector bank.',1650.00,1650.00),
(5,'ICICIBANK','ICICI Bank Ltd','Banking','Private sector bank.',1200.00,1200.00),
(6,'SBIN','State Bank of India','Banking','Public sector bank.',800.00,800.00),
(7,'ITC','ITC Ltd','FMCG','FMCG, hotels, agri-business.',430.00,430.00),
(8,'LT','Larsen & Toubro Ltd','Infrastructure','Engineering and construction.',3600.00,3600.00),
(9,'BHARTIARTL','Bharti Airtel Ltd','Telecom','Telecom services.',1400.00,1400.00),
(10,'HINDUNILVR','Hindustan Unilever Ltd','FMCG','Consumer goods.',2400.00,2400.00),
(11,'WIPRO','Wipro Ltd','IT','IT services and consulting.',480.00,480.00),
(12,'TATAMOTORS','Tata Motors Ltd','Automobile','Passenger and commercial vehicles.',950.00,950.00),
(13,'MARUTI','Maruti Suzuki India Ltd','Automobile','Passenger cars.',12500.00,12500.00),
(14,'ASIANPAINT','Asian Paints Ltd','Consumer','Decorative paints.',2900.00,2900.00),
(15,'SUNPHARMA','Sun Pharmaceutical Industries','Pharma','Generic and specialty pharma.',1700.00,1700.00);

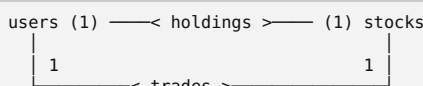
-- priya: pre-built history. Math: 100000 -38000 -29000 -20000 +8600 = 21600 (matches users.cash_balance)
INSERT INTO trades (user_id, stock_id, trade_type, quantity, price, total_amount, realized_pnl, executed_at) VALUES
(3,2,'BUY',10,3800.00,38000.00,0.00,'2026-10-01 10:00:00'),
(3,3,'BUY',20,1450.00,29000.00,0.00,'2026-10-01 10:05:00'),
(3,7,'BUY',50,400.00,20000.00,0.00,'2026-10-02 11:00:00'),
(3,7,'SELL',20,430.00,8600.00,600.00,'2026-10-03 14:30:00');

INSERT INTO holdings (user_id, stock_id, quantity, avg_buy_price) VALUES
(3,2,10,3800.0000),
(3,3,20,1450.0000),
(3,7,30,400.0000);

```

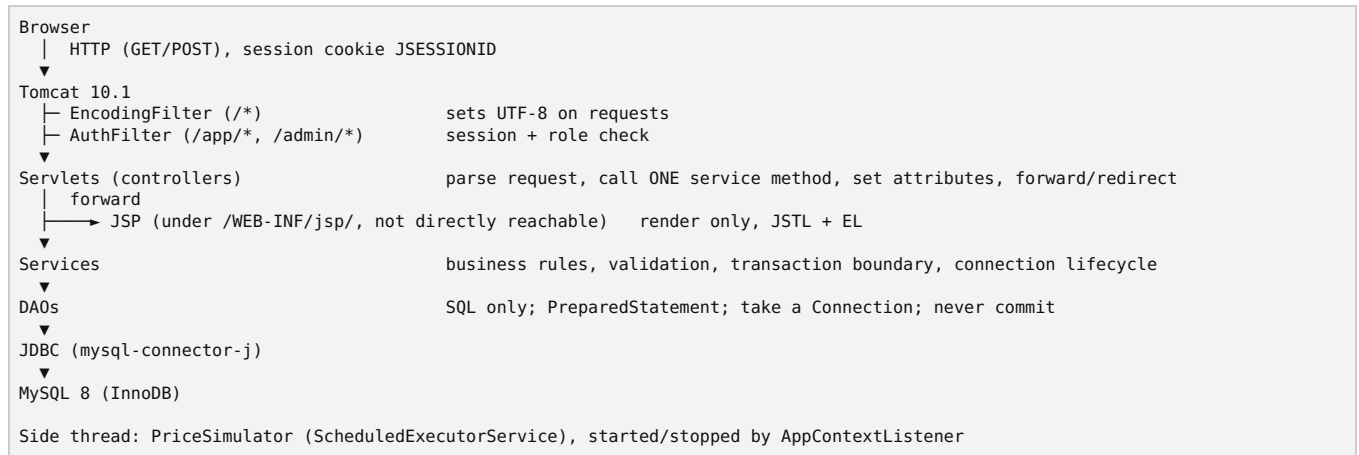
db/reset.sql = schema.sql then seed.sql. Run mysql -u root -p < db/schema.sql && mysql -u root -p < db/seed.sql any time the data gets messy. Do this before every demo rehearsal.

14. ERD Description



- One user has many holdings (0..n); one stock appears in many users' holdings.
- `holdings` is the associative table resolving the many-to-many between users and stocks, with attributes `quantity` and `avg_buy_price`.
- One user has many trades; one stock appears in many trades. Trades are append-only.
- Invariant (enforced by service code, not by DB): for each (user, stock), `holdings.quantity = Σ BUY qty - Σ SELL qty over trades`. Use this as a test.

15. System Architecture

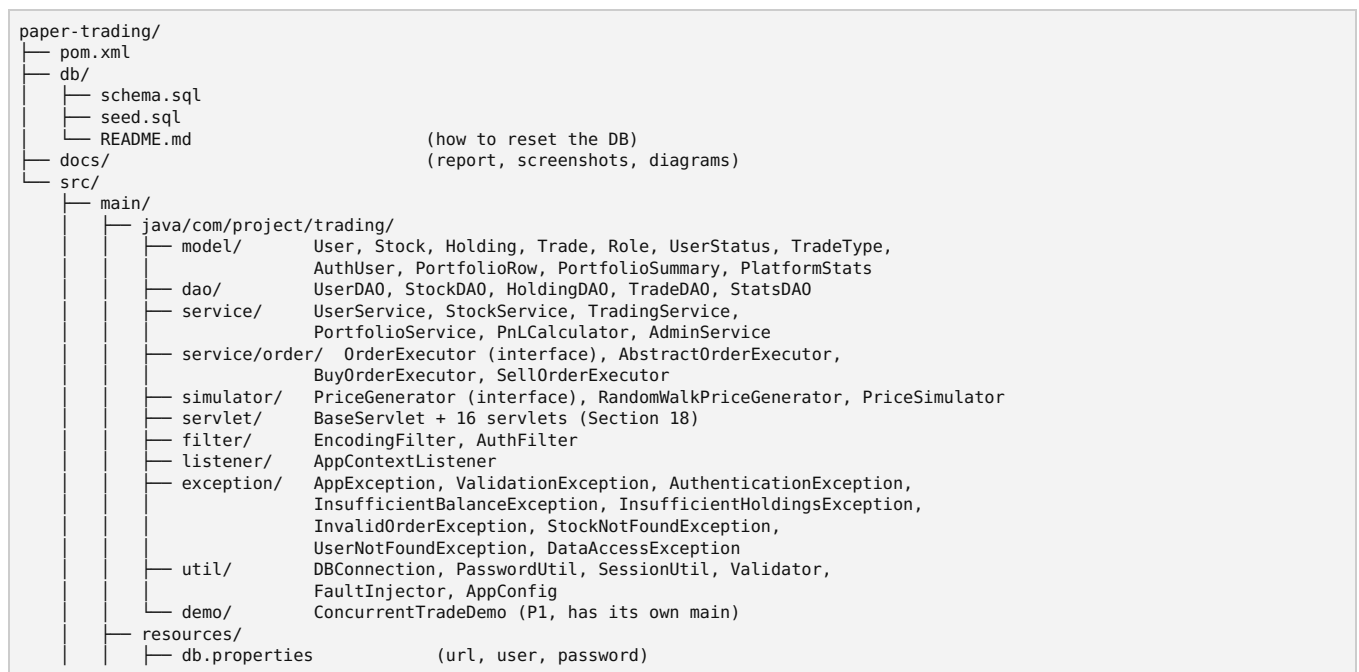


Responsibilities, one line each:

Layer	Does	Must NOT do
Model	Hold data, getters/setters, no logic beyond trivial	Touch DB or HTTP
Servlet	Read params, call service, put results in request scope, forward/redirect, translate exceptions to messages	Contain SQL or business rules
JSP	Display (<code><c:forEach></code> , <code><c:out></code> , <code><fmt:formatNumber></code>)	Scriptlets (<code><% %></code>), SQL, business logic
Service	Validate, enforce rules, open <code>Connection</code> , define transaction	Know about <code>HttpServletRequest</code>
DAO	Execute SQL, map <code>ResultSet</code> ↔ model	Open/close connections, commit, validate business rules
Util	<code>DBConnection</code> (reads <code>db.properties</code>), <code>PasswordUtil</code> , <code>SessionUtil</code> , <code>Validator</code> , <code>FaultInjector</code> , <code>AppConfig</code> (constants such as <code>INITIAL_BALANCE</code>)	Hold request state

Servlet thread-safety rule: Tomcat uses **one servlet instance for all concurrent requests**. Never store per-request data (user, form values) in servlet instance fields. Locals only. This is a classic exam question; you can say you followed it.

16. Java Package Structure



db.properties.example	(committed; db.properties is .gitignored)
webapp/	
index.jsp	(redirect to /login)
css/style.css	(ONE stylesheet)
js/app.js	(optional: confirm dialogs only)
WEB-INF/	
web.xml	(session timeout, error pages, welcome file)
jsp/	
fragments/	header.jspf, footer.jspf
login.jsp, register.jsp, dashboard.jsp, market.jsp,	
stock-detail.jsp, portfolio.jsp, history.jsp, profile.jsp, error.jsp	
admin/	dashboard.jsp, users.jsp, stocks.jsp, prices.jsp, trades.jsp
test/java/com/project/trading/	PnLCalculatorTest, PasswordUtilTest (P1)

About 60 classes, but most are 20-60 lines. Total estimate: 3,500-4,500 lines of Java + JSP across four people. Putting JSPs under WEB-INF means they can only be reached through a servlet forward, so nobody can bypass authentication by typing a JSP URL.

17. Class Responsibilities

Models (frozen by the Leader in the first 2.5 hours)

POJOs: private fields, public getters/setters, no-arg constructor. Money fields are `BigDecimal`.

Class	Fields
User	long id; String username, email, fullName, passwordHash; Role role; UserStatus status; BigDecimal cashBalance; Timestamp createdAt
Stock	long id; String symbol, companyName, sector, description; BigDecimal price, prevPrice; boolean active; Timestamp updatedAt; helper getChangePercent()
Holding	long userId, stockId; int quantity; BigDecimal avgBuyPrice; Stock stock (stock is filled by the join query)
Trade	long id, userId, stockId; TradeType type; int quantity; BigDecimal price, totalAmount, realizedPnl; Timestamp executedAt; String username, stockSymbol (last two filled by join queries)
AuthUser	long id; String username; Role role immutable; the only thing stored in the session (never the password hash)
PortfolioRow	symbol, companyName, quantity, avgBuyPrice, currentPrice, invested, currentValue, unrealizedPnl, unrealizedPct, stockId
PortfolioSummary	List<PortfolioRow> rows; cash, totalInvested, totalCurrentValue, totalUnrealizedPnl, realizedPnl, netWorth, overallPnl
PlatformStats	int totalUsers, activeStocks, totalTrades; BigDecimal totalTradedValue; Map<String,Integer> topStocksByTrades
Role, UserStatus, TradeType	enums

DAO contract (frozen signatures; every method takes `Connection c` first and throws `SQLException`)

DAO (owner)	Methods
UserDAO (Dev 2)	User findById(c, id) · User findByIdForUpdate(c, id) · User findByUsername(c, username) · boolean existsByUsername(c, u) · boolean existsByEmail(c, e) · long insert(c, User) · void updateProfile(c, id, fullName, email) · void updatePasswordHash(c, id, hash) · void updateUser(c, id, UserStatus) · void adjustCash(c, id, BigDecimal delta) · List<User> findAll(c)
StockDAO (Dev 3)	Stock findById(c, id) · Stock findBySymbol(c, symbol) · List<Stock> findAllActive(c) · List<Stock> search(c, q) (active only, LIKE ? on symbol and name) · List<Stock> findAll(c) · long insert(c, Stock) · void update(c, Stock) · void setActive(c, id, boolean) · void updatePrice(c, id, BigDecimal) · void updatePricesBatch(c, Map<Long,BigDecimal>)
HoldingDAO (Dev 1)	Holding find(c, uid, sid) · Holding findForUpdate(c, uid, sid) · void insert(c, Holding) · void update(c, uid, sid, qty, avg) · void delete(c, uid, sid) · List<Holding> findByUserWithStock(c, uid) (JOIN stocks)
TradeDAO (Dev 1)	long insert(c, Trade) · List<Trade> findByUser(c, uid, limit) · List<Trade> findRecent(c, limit) (JOIN users+stocks) · BigDecimal sumRealizedPnl(c, uid)
StatsDAO (Dev 2)	PlatformStats load(c) (aggregate queries: COUNT, SUM, GROUP BY with LIMIT 5)

Bold = needed by the trade transaction; their owners deliver them **first** (by end of Day 1 morning).

StockDAO.updatePrice SQL, **order matters in MySQL**:

```
UPDATE stocks SET prev_price = price, price = ? WHERE id = ?
```

MySQL evaluates SET assignments left to right, so prev_price captures the old price. Reverse the order and prev_price equals the new price. Verified.

Service contract (frozen signatures)

Service (owner)	Methods
UserService (Dev 2)	User register(username, email, fullName, password) throws ValidationException · User login(username, password) throws AuthenticationException · User getById(id) throws UserNotFoundException · void updateProfile(id, fullName, email) throws ValidationException · void changePassword(id, oldPw, newPw) throws ValidationException,

	AuthenticationException · List<User> listAll() · void setBlocked(adminId, targetId, boolean blocked) throws ValidationException, UserNotFoundException
StockService (Dev 3)	List<Stock> listActive() · List<Stock> search(q) · Stock getById(id) throws StockNotFoundException · List<Stock> listAll() · Stock add(symbol, name, sector, desc, price) throws ValidationException · void update(id, name, sector, desc) throws ... · void setActive(id, boolean) · void updatePrice(id, BigDecimal) throws ValidationException, StockNotFoundException
TradingService (Dev 1)	Trade buy(userId, stockId, qty) throws AppException · Trade sell(userId, stockId, qty) throws AppException · List<Trade> getHistory(userId, limit) · List<Trade> getRecentTrades(limit)
PortfolioService (Dev 4)	PortfolioSummary getSummary(userId) (loads holdings via HoldingDAO.findByIdWithStock, cash via UserDAO.findById, realized via TradeDAO.sumRealizedPnL, computes via PnLCalculator)
AdminService (Dev 2)	PlatformStats getPlatformStats()

Order execution (the OOP that is actually justified)

```
interface OrderExecutor { Trade execute(Connection c, long userId, long stockId, int qty)
    throws AppException, SQLException; }

abstract class AbstractOrderExecutor implements OrderExecutor {
    // protected: userDAO, stockDAO, holdingDAO, tradeDAO
    // protected: loadTradableUser(c, userId)      -> lock + status/role checks
    // protected: loadStock(c, stockId)            -> existence check
    // protected: recordTrade(c, ...)              -> tradeDAO.insert + FaultInjector hook
}
class BuyOrderExecutor extends AbstractOrderExecutor { execute(...) { buy steps } }
class SellOrderExecutor extends AbstractOrderExecutor { execute(...) { sell steps } }
```

TradingService.buy() = placeOrder(new BuyOrderExecutor(), ...); sell() = placeOrder(new SellOrderExecutor(), ...). One transaction template, two behaviours chosen through an interface: inheritance (shared checks in the abstract base), polymorphism (the service calls `execute` without knowing which), interface (the contract).

Other classes

- BaseServlet extends HttpServlet: helpers `currentUser(req)`, `forward(req, resp, jsp)`, `redirect(req, resp, path)`, `parseLongParam(req, name)`, `flashSuccess/flashError`. Every servlet extends it (genuine inheritance).
- AuthFilter: no AuthUser in session → redirect `/login` with flash. `/admin/*` and role ≠ ADMIN → `sendError(403)`. `/app/*` and role = ADMIN → redirect `/admin/dashboard`.
- SessionUtil: `login(req, user)` (calls `req.changeSessionId()` against session fixation, then stores `AuthUser`), `logout(req)`, `getAuthUser(req)`, `flash get-and-remove`.
- Validator: `requireNotBlank`, `matches(regex)`, `isValidEmail`, `isValidPassword`, `parsePositiveInt`. Static methods throw `ValidationException`.
- PasswordUtil: `String hash(char[] pw) → salt:hash`; `boolean verify(char[] pw, String stored)` using `SecureRandom` + `PBKDF2WithHmacSHA256`, 65,536 iterations, 256-bit; constant-time compare via `MessageDigest.isEqual`.
- DBConnection: static initialiser loads `db.properties` from the classpath; `getConnection()` returns `DriverManager.getConnection(url, user, pass)`. URL: `jdbc:mysql://localhost:3306/papertrade?useUnicode=true&characterEncoding=UTF-8&serverTimezone=Asia/Kolkata`.
- FaultInjector: static volatile boolean `failMidTrade`; static void `checkpoint(String where)`.
- PriceGenerator (interface): `BigDecimal next(BigDecimal current)`. `RandomWalkPriceGenerator`: multiply by `1 + uniform(-0.015, +0.015)`, round to 2 dp, floor at ₹1.00.
- PriceSimulator: wraps `ScheduledExecutorService` (1 daemon thread), `AtomicBoolean running`, `start(intervalSec)`, `stop()`, `isRunning()`. Each tick: one connection, `StockDAO.findAllActive`, compute new prices, `updatePricesBatch`, `commit`. catch (Throwable t) inside the task, because an uncaught exception **silently cancels all future runs** of a scheduled task. This is the classic bug.
- AppContextListener implements `ServletContextListener`: `contextInitialized` creates the `PriceSimulator` (not started) and stores it in `ServletContext`; `contextDestroyed` calls `shutdownNow()`. Without the shutdown, Tomcat redeploys leak threads.

18. Servlet / API Mapping

All servlets use `@WebServlet`. Session timeout (15 min), error pages (403/404/500/Throwable → `/WEB-INF/jsp/error.jsp`) go in `web.xml`.

URL	Servlet	Methods	Auth	Owner	Service call	Result
/ (index.jsp)	-	GET	public	Dev 1	-	redirect <code>/login</code>
/login	LoginServlet	GET, POST	public	Dev 2	<code>userService.login</code>	forward <code>login.jsp</code> / redirect by role
/register	RegisterServlet	GET, POST	public	Dev 2	<code>userService.register</code>	forward <code>register.jsp</code> / redirect <code>/login</code>
/logout	LogoutServlet	POST (GET too, for simplicity)	any	Dev 2	-	invalidate, redirect <code>/login</code>
/app/dashboard	DashboardServlet	GET	USER	Dev 4	<code>portfolioService.getSummary</code> , <code>tradingService.getHistory(uid,5)</code>	<code>dashboard.jsp</code>
/app/market	MarketServlet	GET (?q=)	USER	Dev 3	<code>stockService.listActive</code> / <code>search</code>	<code>market.jsp</code>
/app/stock	StockDetailServlet	GET (?id=)	USER	Dev 3	<code>stockService.getById</code> , holding qty, cash	<code>stock-detail.jsp</code>
/app/buy	BuyServlet	POST (stockId)	USER	Dev 1	<code>tradingService.buy</code>	success → redirect

		quantity)				/app/portfolio; error → flash + redirect /app/stock?id=
/app/sell	SellServlet	POST (stockId, quantity)	USER	Dev 1	tradingService.sell	same
/app/portfolio	PortfolioServlet	GET	USER	Dev 4	portfolioService.getSummary	portfolio.jsp
/app/history	HistoryServlet	GET	USER	Dev 4	tradingService.getHistory(uid,100)	history.jsp
/app/profile	ProfileServlet	GET, POST (action = profile or password)	USER	Dev 2	userService.*	profile.jsp
/admin/dashboard	AdminDashboardServlet	GET	ADMIN	Dev 2	adminService.getPlatformStats	admin/dashboard.jsp
/admin/users	AdminUsersServlet	GET; POST (userId, blocked)	ADMIN	Dev 2	userService.listAll/setBlocked	admin/users.jsp
/admin/stocks	AdminStocksServlet	GET (?edit=); POST (action = add, edit or toggle)	ADMIN	Dev 3	stockService.*	admin/stocks.jsp
/admin/prices	AdminPricesServlet	GET; POST (action = price, simulator or fault)	ADMIN	Dev 3	stockService.updatePrice, PriceSimulator, FaultInjector	admin/prices.jsp
/admin/trades	AdminTradesServlet	GET	ADMIN	Dev 2	tradingService.getRecentTrades(200)	admin/trades.jsp

Conventions that remove integration bugs (Leader enforces):

- Every state-changing action is **POST**. Every success is followed by **redirect**. Every GET only reads.
- Forward target is always a path under /WEB-INF/jsp/.
- Request attribute names are fixed per page (Section 19).
- Flash messages live in the session under flashSuccess / flashError; header.jspf prints and removes them.
- All links use \${pageContext.request.contextPath}.

Exception → message mapping in BuyServlet / SellServlet:

Exception	Message shown
InvalidOrderException, ValidationException	its own message
InsufficientBalanceException	"Insufficient funds. Available ₹X, required ₹Y."
InsufficientHoldingsException	"You hold only N shares."
StockNotFoundException	"Stock not found."
DataAccessException	"Trade failed. No changes were saved." (log the cause to the console)

19. UI / Page Specification

Layout: one header.jspf (banner, navbar chosen by role, flash messages) and one footer.jspf. One style.css. No Bootstrap CDN: **the demo must work offline**. If you want Bootstrap, download the min CSS once into webapp/css/.

Number format: <fmt:formatNumber value="{x}" type="number" minFractionDigits="2" maxFractionDigits="2" groupingUsed="true"/> prefixed with ₹. It will show 1,00,000 as 100,000 (Western grouping). Do not waste time on Indian grouping.

Every JSP starts with <%@ page contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %> and uses <%@ taglib prefix="c" uri="jakarta.tags.core" %> (JSTL 3.0 URIs; the old http://java.sun.com/jsp/jstl/core fails on Tomcat 10 + JSTL 3). Escape all user data with <c:out>.

#	Screen	Endpoint	Request attrs (JSP reads)	Inputs / actions	DB interaction	Validation	Error states
1	Login	/login	error, username	username, password; link to register	users SELECT by username	non-blank	"Invalid username or password" (same for unknown/wrong); "Account blocked"
2	Register	/register	errors (List), form values	username, email, full name, password, confirm	SELECT exists ×2, INSERT users	FR-01/02; confirm matches	per-field list; duplicate username/email
3	Dashboard	/app/dashboard	summary, recentTrades	links to market/portfolio	users, holdings+stocks, trades	-	DB error → error.jsp
4	Market	/app/market	stocks, q	search box (GET)	stocks SELECT (+ LIKE ?)	trim q, max 50 chars	"No stocks match"
5	Stock detail	/app/stock?id=	stock, heldQty, cash	Buy form, Sell form	stocks, holdings, users SELECT	id numeric	unknown id → 404 page
6	Buy (form on #5)	POST /app/buy	-	stockId, quantity	TX (Section 11)	int 1-10,000	insufficient funds; inactive; blocked; DB failure
7	Sell (form on #5)	POST /app/sell	-	stockId, quantity	TX	int 1-10,000	insufficient holdings; unknown stock

8	Portfolio	/app/portfolio	summary	"Trade" link per row → stock detail	holdings JOIN stocks, users, SUM trades	-	empty portfolio message
9	History	/app/history	trades	-	trades JOIN stocks, LIMIT 100	-	empty message
10	Profile	/app/profile	user, errors	edit name/email; change password	UPDATE users	name 1-100; email format+unique; password rules + old password check	wrong old password; duplicate email
11	Logout	/logout	-	-	-	-	-
A1	Admin login	/login (shared)	-	-	-	-	-
A2	Admin dashboard	/admin/dashboard	stats	-	COUNT/SUM/GROUP BY	-	-
A3	User management	/admin/users	users	Block/Unblock	users SELECT, UPDATE status	not self, not ADMIN	"Cannot block admin"
A4	Stock management	/admin/stocks	stocks, editStock	add/edit form, toggle active	stocks INSERT/UPDATE	symbol [A-Z0-9&-]{1,15} unique; price > 0	duplicate symbol
A5	Price update	/admin/prices	stocks, simulatorRunning, faultOn	per-row price field; simulator toggle; fault toggle	UPDATE stocks	price > 0, ≤ 9,999,999.99	invalid number
A6	Trade monitor	/admin/trades	trades	-	trades JOIN users+stocks, LIMIT 200	-	-

Plus `error.jsp` (shared by 403/404/500). That is **14 JSPs + 2 fragments**.

Buy/Sell on the stock detail page (screens 6 and 7) are two small forms, not two pages. A "confirmation page" would be one more servlet, one more JSP and one more bug. Cut.

20. Authentication & Authorization

Registration: `UserService.register` validates, checks duplicates (`existsByUsername`, `existsByEmail`), hashes with `PasswordUtil`, inserts. The DB default supplies ₹100,000. Also catch `SQLIntegrityConstraintViolationException` (error 1062) as the race-safe duplicate fallback and convert it into a `ValidationException`.

Login: `findByUsername` → `PasswordUtil.verify` → status check → `SessionUtil.login` (`request.changeSessionId()`), then store `AuthUser` under a single session key). Redirect: ADMIN → `/admin/dashboard`, USER → `/app/dashboard`. Same message for unknown user and wrong password, so usernames cannot be enumerated.

Logout: `session.invalidate()`, redirect `/login`.

Session: `<session-config><session-timeout>15</session-timeout></session-config>` in `web.xml`. Expired session → `AuthFilter` sees no `AuthUser` → redirect to `/login` with "Session expired. Please log in again."

Authorization (`AuthFilter`, mapped to `/app/*` and `/admin/*`):

Situation	Result
No <code>AuthUser</code>	redirect <code>/login</code>
<code>/admin/*</code> and role ≠ ADMIN	HTTP 403 → <code>error.jsp</code>
<code>/app/*</code> and role = ADMIN	redirect <code>/admin/dashboard</code>

Blocked-after-login: the Filter does not hit the DB. Instead the trade transaction reads the user row anyway (for `FOR UPDATE`) and rejects BLOCKED users. Cheap and correct.

Security checklist (all implementable in 3 days)

Measure	Where
<code>PreparedStatement</code> everywhere, zero SQL string concatenation (including <code>LIKE ?</code> with <code>"%"+q+"%"</code> bound as a parameter)	all DAOs
Server-side validation of every field	<code>Validator</code> , services
PBKDF2 salted hashing, constant-time compare	<code>PasswordUtil</code>
Role checks on every protected URL	<code>AuthFilter</code>
Session id regenerated at login; session invalidated at logout	<code>SessionUtil</code>
Output escaping	<code><c:out></code> / <code>fn:escapeXml</code> in all JSPs
JSPs unreachable directly	<code>WEB-INF/jsp</code>
Generic error pages, no stack traces to the browser	<code>web.xml</code> error pages
DB credentials in <code>db.properties</code> , not in source; file ignored by git	resources

Not implemented, stated honestly in the Limitations section of the report: CSRF tokens, login rate limiting, HTTPS, account lockout, password reset. CSRF tokens are P2; add only if everything else is done.

21. Exception Handling

Exception	Type	Thrown by	Handled in	Shown as
AppException	checked, base	-	-	base for all business exceptions
ValidationException	checked	Validator, UserService, StockService	servlets	field/form messages
AuthenticationException	checked	UserService.login, changePassword	LoginServlet, ProfileServlet	"Invalid username or password"
InsufficientBalanceException	checked, carries available, required	BuyOrderExecutor	BuyServlet	red flash
InsufficientHoldingsException	checked, carries held, requested	SellOrderExecutor	SellServlet	red flash
InvalidOrderException	checked	TradingService, executors	order servlets	red flash
StockNotFoundException	checked	StockService, executors	servlets	flash / 404
UserNotFoundException	checked	UserService, executors	servlets	error page
DataAccessException	unchecked (RuntimeException) wrapping SQLException	services	web.xml error page or order servlets	"No changes were saved"

Rule: DAOs throw `SQLException`; services translate into `DataAccessException`. Services throw checked business exceptions. Servlets catch them and turn them into messages. Nothing swallows an exception silently. `rollbackQuietly` may swallow the rollback's own failure but must print it.

22. Java Concepts Mapping

Concept	Where used	Why it exists	Relevant classes
Encapsulation	All models: private fields, getters/setters, immutable <code>AuthUser</code> ; <code>PriceSimulator</code> hides its executor	Control access to state	<code>User</code> , <code>Stock</code> , <code>Holding</code> , <code>Trade</code> , <code>AuthUser</code> , <code>PriceSimulator</code>
Inheritance	<code>AbstractOrderExecutor</code> ← <code>Buy/Sell</code> ; <code>BaseServlet</code> ← all servlets; <code>AppException</code> ← business exceptions	Share validation and helpers without copy-paste	<code>AbstractOrderExecutor</code> , <code>BuyOrderExecutor</code> , <code>SellOrderExecutor</code> , <code>BaseServlet</code> , <code>AppException</code> hierarchy
Polymorphism	<code>TradingService</code> invokes <code>OrderExecutor.execute</code> without knowing which one; <code>PriceSimulator</code> invokes <code>PriceGenerator.next</code> ; catch (<code>AppException</code> e) treats subclasses uniformly	One transaction template, interchangeable behaviours	<code>TradingService</code> , <code>OrderExecutor</code> , <code>PriceGenerator</code>
Interfaces	<code>OrderExecutor</code> , <code>PriceGenerator</code> , plus JDK interfaces implemented: <code>ServletContextListener</code> , <code>Filter</code> , <code>Runnable/Callable</code>	Define contracts that implementations can vary behind	<code>OrderExecutor</code> , <code>PriceGenerator</code> , <code>AppContextListener</code> , <code>AuthFilter</code>
Abstract class	<code>AbstractOrderExecutor</code>	Common steps (lock user, check status) shared by both order types	<code>AbstractOrderExecutor</code>
Collections	<code>List<Stock></code> , <code>List<Trade></code> ; <code>Map<String,Integer></code> for top stocks; <code>Map<Long,BigDecimal></code> for batch price updates; <code>Comparator</code> sorting in market page / portfolio rows	Natural result sets and lookups	<code>StockDAO</code> , <code>StatsDAO</code> , <code>PortfolioService</code> , <code>PlatformStats</code>
Generics	Typed collections; optionally <code>Map<Long,BigDecimal></code>	Type safety	throughout
Enums	<code>Role</code> , <code>UserStatus</code> , <code>TradeType</code> mapped to MySQL ENUM	Closed value sets	model package
Exception handling	try/catch/finally, try-with-resources for <code>Connection/PreparedStatement/ResultSet</code> , multi-catch	Reliable resource and error handling	all services, DAOs
Custom exceptions	Section 21	Business errors that map cleanly to UI messages	exception package
Multithreading	<code>ScheduledExecutorService</code> price updates; <code>AtomicBoolean</code> ; volatile flag in <code>FaultInjector</code> ; <code>ConcurrentTradeDemo</code> using <code>ExecutorService</code> ; servlets running concurrently on Tomcat's pool	Genuine: periodic background job, and concurrent users trading	<code>PriceSimulator</code> , <code>AppContextListener</code> , <code>FaultInjector</code> , <code>ConcurrentTradeDemo</code>
JDBC	<code>DriverManager</code> , <code>PreparedStatement</code> , <code>ResultSet</code> , <code>getGeneratedKeys</code> , batch update	Persistence	<code>DBConnection</code> , all DAOs
Transactions	<code>setAutoCommit(false)</code> , <code>commit</code> , <code>rollback</code> , <code>SELECT ... FOR UPDATE</code> , InnoDB	Atomic buy/sell	<code>TradingService</code> , executors
DAO pattern	SQL isolated in <code>*DAO</code>	Separation of concerns	dao package
Servlets	<code>@WebServlet</code> , <code>doGet/doPost</code> , <code>RequestDispatcher.forward</code> , <code>sendRedirect</code> , PRG	HTTP handling	servlet package
Filters	<code>AuthFilter</code> , <code>EncodingFilter</code>	Cross-cutting auth/encoding	filter package
Listeners	<code>ServletContextListener</code>	App lifecycle (start/stop thread)	<code>AppContextListener</code>
Sessions	<code>HttpSession</code> , <code>AuthUser</code> , flash messages, timeout, <code>changeSessionId</code>	Stateful login over stateless HTTP	<code>SessionUtil</code> , <code>AuthFilter</code>

JSP/JSTL/EL	<c:forEach>, <c:if>, <c:out>, <fmt:formatNumber>	View rendering without scriptlets	WEB-INF/jsp/*
BigDecimal	All money	Exactness	models, PnLCalculator, executors

Honest note on concepts you will be tempted to claim: "multiple inheritance via interfaces", "design patterns: Strategy, DAO, Template Method". Strategy (executors), DAO and Template Method (placeOrder) are real here and fair to mention. Do not name-drop Singleton or Factory unless you actually wrote one.

23. Testing Plan

Manual test cases executed against the seeded DB (reset.sql before each full pass). Use two browsers (or one normal + one private window) for concurrency and role tests. Simulator OFF unless stated.

ID	Scenario	Input	Expected result	Pri
Registration				
REG-01	Valid registration	sneha, sneha@x.com, "Sneha R", Test@1234	Redirect to login, flash success; DB row has ₹100,000.00, role USER, hash ≠ plaintext	P0
REG-02	Duplicate username	rahul	Error "Username already taken"; no new row	P0
REG-03	Duplicate email	rahul@papertrade.local	Error "Email already registered"	P0
REG-04	Weak password	abc	Error listing password rule	P0
REG-05	Invalid email / blank fields	foo, blank	Field errors; entered values preserved except password	P0
REG-06	XSS in name	<script>alert(1)</script> as full name	Stored; displayed as literal text, no alert	P0
REG-07	SQL injection in username	' OR '1'='1	Rejected by regex; no DB error	P0
Login / session				
LOG-01	Valid user login	rahul / Demo@123	Redirect /app/dashboard	P0
LOG-02	Valid admin login	admin / Admin@123	Redirect /admin/dashboard	P0
LOG-03	Wrong password	rahul / wrong	"Invalid username or password"	P0
LOG-04	Unknown user	nobody / x	Same message as LOG-03	P0
LOG-05	Blocked user	Admin blocks rahul, then rahul logs in	"Account blocked"; no session	P0
LOG-06	Logout then Back button	Logout, press Back, click a link	Redirected to login	P0
LOG-07	Session expiry	Set timeout to 1 min temporarily; wait; click	Redirect to login with "Session expired"	P1
LOG-08	Session id changes at login	Compare JSESSIONID before/after login	Different value	P1
Authorization				
AUTH-01	Anonymous hits /app/portfolio	direct URL	Redirect to login	P0
AUTH-02	Anonymous hits /admin/users	direct URL	Redirect to login	P0
AUTH-03	USER hits /admin/dashboard	logged in as rahul	HTTP 403 page	P0
AUTH-04	ADMIN hits /app/market	logged in as admin	Redirect to /admin/dashboard	P0
AUTH-05	Direct JSP URL	/WEB-INF/jsp/login.jsp, /jsp/login.jsp	404	P0
AUTH-06	Block self / other admin	admin POSTs block for admin id	Rejected with message	P0
AUTH-07	Blocked user with live session tries to buy	rahul logged in, admin blocks, rahul buys	Rejected "Account cannot trade"	P1
Buy				
BUY-01	Valid buy	rahul buys 10 TCS @3900	Cash 61,000.00; holding 10 @3900.0000; BUY trade total 39,000.00; redirect to portfolio	P0
BUY-02	Second buy updates average	then 10 TCS after admin sets price 4100	Qty 20, avg 4000.0000, cash 20,000.00	P0
BUY-03	Insufficient balance	buy 100 TCS (390,000)	InsufficientBalanceException message; cash/holdings/trades unchanged	P0
BUY-04	Exactly all cash	buy 25 SBIN @800 = 20,000 when cash is 20,000	Succeeds, cash 0.00	P0
BUY-05	Invalid quantity	0, -5, abc, 1.5, empty, 10001	Validation error each time; no DB change	P0
BUY-06	Invalid stock	stockId=9999, stockId=abc	"Stock not found" / validation error	P0
BUY-07	Inactive stock	admin deactivates stock; user buys	"Stock not available for trading"	P0
BUY-08	Refresh after success	press F5 on portfolio page	No second purchase (PRG)	P0
BUY-09	Price changed since page load	open stock page; admin changes price; submit	Executes at the new price; trade row shows it	P1
Sell				
SELL-01	Valid partial sell	priya sells 10 INFY @1500 (avg 1450)	Qty 10, cash +15,000, realized +500.00	P0
SELL-02	Sell everything	priya sells all 30 ITC	Holding row deleted (not qty 0); trade recorded	P0
SELL-03	Insufficient holdings	sell 31 ITC while holding 30	InsufficientHoldingsException; no change	P0

SELL-04	Sell stock never owned	rahul sells RELIANCE	"You hold only 0 shares"	P0
SELL-05	Invalid quantity	0, -1, abc	Validation error	P0
SELL-06	Sell deactivated stock	priya holds stock, admin deactivates, priya sells	Allowed	P1
SELL-07	Average unchanged by sell	after SELL-01	avg still 1450.0000	P0
P&L				
PNL-01	Seed data check	login priya	Net worth 103,500.00; unrealized 2,900.00; realized 600.00; overall 3,500.00	P0
PNL-02	Price change reflects	admin sets TCS 4000	TCS unrealized = $10 \times (4000 - 3800) = 2,000$; totals update	P0
PNL-03	Invariant	after several trades	overall P&L = realized + unrealized ($\pm ₹1$)	P0
PNL-04	Holdings vs ledger	after tests	for each holding, $qty = \Sigma \text{BUY} - \Sigma \text{SELL}$ (SQL check)	P1
Failure / transactions				
TX-01	Injected failure on BUY	enable fault toggle; buy 10 TCS	Error "no changes were saved"; cash, holdings, trades identical to before (check in MySQL)	P0
TX-02	Injected failure on SELL	enable toggle; sell	same, no change	P0
TX-03	DB down during trade	stop MySQL service, then buy	Friendly error page/flash, no stack trace to browser; app recovers after MySQL restart	P1
TX-04	CHECK as backstop	in SQL: UPDATE users SET cash_balance = -1 WHERE id=2	MySQL error 3819	P1
Concurrency				
CON-01	ConcurrentTradeDemo	10 threads $\times$ buy 15 RELIANCE, fresh rahul	Exactly 2 succeed, 8 insufficient, cash 14,500.00, never negative	P1
CON-02	Double-click Buy	click twice fast with amount costing > half of cash	At most one succeeds when cash covers only one	P1
CON-03	Price simulator + trading	simulator ON (5 s), trade 5 times	No deadlock, no exceptions in console	P1
Admin				
ADM-01	Add stock	ZOMATO, ₹200	Appears in market	P0
ADM-02	Duplicate symbol	TCS	Validation error	P0
ADM-03	Price ≤ 0 or non-numeric	0, -5, abc	Rejected	P0
ADM-04	Update price	TCS $\rightarrow$ 4000	price=4000, prev_price=3900; market shows +2.56%	P0
ADM-05	Deactivate stock	toggle	Hidden from market; user's holding still visible	P0
ADM-06	Trade monitor	after trades	All users' trades visible newest first	P0
ADM-07	Stats	after trades	Counts and totals match SQL COUNT/SUM	P0
UI/encoding				
UI-01	₹ symbol	any page	Renders as ₹, not ?/â, ¹	P0

Deliverable: a spreadsheet with these IDs and Pass/Fail plus screenshot for the P0 failures that were fixed. Examiners like a test report that shows bugs found and fixed.

24. Team Responsibilities

The suggested split was revised. The original had Dev 1 owning all trading code **and** needing Dev 2's UserDAO and Dev 3's StockDAO. The revision fixes ownership by **file**, not by feature, and puts the two DAOs on which trading depends at the top of their owners' queues.

Dev	Role	Owens (files nobody else edits)	Ready-by deadlines others rely on
1 — Leader	Architecture, trading core, integration, review	pom.xml, web.xml, db/*, model/*, exception/*, util/DBConnection, AppConfig, SessionUtil, Validator, FaultInjector, BaseServlet, HoldingDAO, TradeDAO, service/order/*, TradingService, BuyServlet, SellServlet, demo/*	Contract freeze (schema, models, exceptions, DBConnection) at hour 2.5 ; BaseServlet/SessionUtil/Validator at hour 4
2	Auth, users, admin oversight	PasswordUtil, UserDAO, UserService, AuthFilter, EncodingFilter, Login/Register/Logout/Profile servlets + JSPs, StatsDAO, AdminService, AdminDashboardServlet, AdminUsersServlet, AdminTradesServlet + JSPs	UserDAO.findByIdForUpdate and adjustCash by hour 4
3	Stocks, market, admin stocks, simulator	StockDAO, StockService, MarketServlet, StockDetailServlet, AdminStocksServlet, AdminPricesServlet + JSPs, simulator/*, ApplicationContextListener	StockDAO.findById by hour 4 ; stock detail page with Buy/Sell forms by end of Day 1
4	Portfolio, history, dashboard, UI	PnLCalculator, PortfolioService, Portfolio/History/Dashboard servlets + JSPs, error.jsp, css/*, fragments/*	style.css + header.jspf + footer.jspf by hour 2 (everyone's JSPs need them)

Why this removes blocking:

- Dev 4 has real data on Day 1 (seeded priya), so portfolio/history do not wait for the buy flow.
- Dev 2 and Dev 3 work against the real schema directly. The Leader's freeze is only schema + models + exceptions + DBConnection, about 2.5 hours, and during those hours Dev 2 writes PasswordUtil, Dev 3 reads the schema and sketches DAO SQL, Dev 4 writes CSS/header/footer. Nobody is idle.
- Dev 1's trade code needs two DAO methods from Dev 2 and one from Dev 3. They are on page one of those developers'

queues.

- Servlets only call service methods whose signatures are frozen in Section 17. Each owner commits an **empty skeleton** of their DAO/service with the exact signatures in the first 3 hours so everyone else compiles.

Git rules (non-negotiable)

- One repository, `main` protected. Each dev works on `dev<N>/<feature>` branches; merge to `main` via Dev 1 only, at least **twice per day** (end of morning, end of day).
- Nobody edits a file they do not own. Need a change? Message the owner. Shared-file changes go through the Leader.
- `db.properties` is git-ignored. Each developer runs their **own local MySQL** loaded from `db/schema.sql` + `seed.sql`. A shared remote DB means four people corrupting each other's test data.
- Commit small and often. Pull before starting work. Resolve conflicts in the IDE, not by overwriting.

Environment standard (decide in the first 30 minutes, same on all four laptops): JDK 17, Maven 3.9, **Tomcat 10.1.x** (not 9, because 9 uses `javax.*` and this code uses `jakarta.*`), MySQL 8.0.x, one IDE configuration. IntelliJ Community has no built-in Tomcat support. Use Eclipse IDE for Enterprise Java (has Tomcat server integration), IntelliJ Ultimate, or the free "Smart Tomcat" plugin for Community. Fallback that always works: `mvn clean package`, copy the WAR into Tomcat's `webapps/`. Verify on Day 1 that everyone can load `http://localhost:8080/<context>/login`.

25. 3-Day Development Plan

Estimates are productive hours with the assumption that people have written at least one servlet before. Planned load is roughly 17-20 hours per person against about 27 available (3 × 9), leaving a buffer for the things that always go wrong.

Day 1: Foundation and vertical slices

Time	Dev 1	Dev 2	Dev 3	Dev 4
0:00-0:45	T01 skeleton, push	Install JDK/Tomcat/MySQL, verify	same	same
0:45-2:30	T02 models, T03 exceptions + <code>DBConnection</code> → freeze #1	T17 <code>PasswordUtil</code>	Read schema; start T26 SQL	T36 CSS + header/footer
2:30-4:00	T04 <code>BaseServlet</code> , <code>SessionUtil</code> , <code>Validator</code> , <code>FaultInjector</code>	T16 <code>UserDAO</code> (tx methods first)	T26 <code>StockDAO</code> (<code>findById</code> first)	T37 <code>PnLCalculator</code>
4:00-6:30	T05 <code>HoldingDAO</code> , T06 <code>TradeDAO</code>	T18 <code>UserService</code> , T19 filters	T27 <code>StockService</code> , T28 market page	T38 <code>PortfolioService</code>
6:30-9:00	T07 <code>OrderExecutor</code> + Buy	T20 login/register/logout + JSPs	T29 stock detail with forms	T39 portfolio page

Deliverables (end of Day 1): all four can run the app; login/register works; market and stock detail render from DB; portfolio renders `priya`'s seeded data; `BuyOrderExecutor` drafted. **Definition of Done:** `main` builds with `mvn clean package`; deployed WAR starts on every laptop; a user can register, log in, see the market and see the portfolio of `priya`. First merge of everything done by Dev 1.

Day 2: Trading and admin

Dev 1	Dev 2	Dev 3	Dev 4
T08 Sell executor, T09 <code>TradingService</code>	T21 profile	T30 admin stocks	T40 history
T10 Buy/Sell servlets, T11 trade tests incl. rollback and overspend	T22 admin dashboard + stats	T31 admin prices + fault toggle	T41 dashboard
Merge + integration	T23 admin users, T24 admin trades	T32 price simulator (P1)	T42 error pages

Deliverables: complete P0 feature set. **Day 2 evening checkpoint (hard gate):** a fresh user can register → buy → see the portfolio update → sell → see history, and the admin can change a price and see the trade. If it is not green, all P1 work is cancelled and Day 3 morning starts with bug fixing, not polish. **Definition of Done:** TX-01 and BUY-01..03, SELL-01..03 pass.

Day 3: Integration, testing, documentation

Block	Work
Morning	Run Section 23 (split: Dev 2 AUTH/LOG/REG; Dev 3 ADM/UI; Dev 4 PNL/BUY/SELL; Dev 1 TX/CON). Fix bugs. T33 sort/colours and T13 concurrency demo only if P0 tests are mostly green.
14:00 FEATURE FREEZE	Only bug fixes after this. Nobody merges a "small improvement".
Afternoon	T43 UI consistency pass; documentation (each dev writes their parts, Section 28); screenshots; T15 final WAR; T46 deploy test on a clean machine; two full demo rehearsals with a timer.

Deliverables: tested WAR, report, screenshots, demo rehearsed twice, DB dump (`mysqldump`) as backup. **Definition of Done:** all P0 tests pass or have documented known issues; demo script completed twice without a surprise.

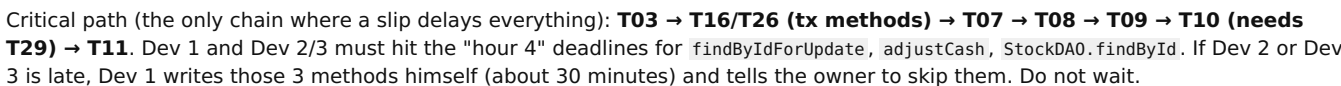
Task board

ID	Task	Owner	Pri	Dependency	Hrs	Deliverable	Day

T01	Maven skeleton, web.xml, .gitignore, db.properties.example, package dirs, load SQL, push	Dev 1	P0	-	0.75	Repo that builds	1
T02	Models, enums, AuthUser, PortfolioRow/Summary, PlatformStats	Dev 1	P0	T01	1	model/*	1
T03	9 exceptions, DBConnection, AppConfig	Dev 1	P0	T01	1	Contract freeze #1	1
T04	BaseServlet, SessionUtil, Validator, FaultInjector	Dev 1	P0	T03	1.5	util/*, BaseServlet	1
T05	HoldingDAO (all methods)	Dev 1	P0	T02, T03	1.5	DAO	1
T06	TradeDAO (all methods)	Dev 1	P0	T02, T03	1	DAO	1
T07	OrderExecutor, AbstractOrderExecutor, BuyOrderExecutor	Dev 1	P0	T05, T06, T16, T26	2	Buy logic	1
T08	SellOrderExecutor	Dev 1	P0	T07	1.25	Sell logic	2
T09	TradingService (placeOrder, buy, sell, history, recent)	Dev 1	P0	T07, T08	1.5	Service	2
T10	BuyServlet, SellServlet	Dev 1	P0	T09, T29	1	Working trades in UI	2
T11	Trade tests (BUY/SELL/TX groups)	Dev 1	P0	T10	2	Test results	2
T12	Daily review + merge (≈1 h × 3)	Dev 1	P0	-	3	Clean main	1-3
T13	ConcurrentTradeDemo	Dev 3	P1	T09	1	Demo class	3
T14	Review architecture/class/sequence docs written by Dev 2	Dev 1	P0	T25	0.5	Reviewed docs	3
T15	Final WAR, demo env, lead two rehearsals	Dev 1	P0	all	2	WAR + DB dump	3
T16	UserDAO (findByIdForUpdate, adjustCash first)	Dev 2	P0	T02, T03	2	DAO	1
T17	PasswordUtil	Dev 2	P0	-	1	Hash/verify	1
T18	UserService	Dev 2	P0	T16, T17, T04	2	Service	1
T19	EncodingFilter, AuthFilter	Dev 2	P0	T04	1	Filters	1
T20	Login/Register/Logout servlets + JSPs	Dev 2	P0	T18, T19, T36	2	Auth flow	1
T21	ProfileServlet + JSP	Dev 2	P0	T18	1.5	Profile	2
T22	StatsDAO, AdminService, admin dashboard + JSP	Dev 2	P0	T06	2	Admin dashboard	2
T23	AdminUsersServlet + JSP	Dev 2	P0	T18	1.5	Block/unblock	2
T24	AdminTradesServlet + JSP	Dev 2	P0	T06	1	Trade monitor	2
T25	AUTH/LOG/REG tests; docs: problem, objectives, scope, FR/NFR, use cases; architecture, class and sequence diagrams (reviewed by Dev 1)	Dev 2	P0	T20	5	Results + report parts	3
T26	StockDAO (findById first)	Dev 3	P0	T02, T03	2	DAO	1
T27	StockService	Dev 3	P0	T26, T04	1.5	Service	1
T28	MarketServlet + market.jsp	Dev 3	P0	T27, T36	2	Market	1
T29	StockDetailServlet + stock-detail.jsp with Buy/Sell forms	Dev 3	P0	T27, T05, T16, T36	2	Stock page	1
T30	AdminStocksServlet + JSP	Dev 3	P0	T27	2	Stock CRUD	2
T31	AdminPricesServlet + JSP (manual price, fault toggle)	Dev 3	P0	T27, T04	1.5	Price update	2
T32	PriceGenerator, RandomWalkPriceGenerator, PriceSimulator, AppContextListener, toggle on prices page	Dev 3	P1	T26, T31	2.5	Simulator thread	2
T33	Market sorting, day-change colours	Dev 3	P1	T28	1	UI	3
T34	ADM/UI tests	Dev 3	P0	T30, T31	1.5	Results	3
T35	Docs: DB schema, ER description, stack, Java concepts table	Dev 3	P0	-	2	Report parts	3
T36	style.css, header.jspf, footer.jspf	Dev 4	P0	-	2	Layout (by hour 2)	1
T37	PnLCalculator (+ JUnit P1)	Dev 4	P0	T02	1	Pure calc	1
T38	PortfolioService	Dev 4	P0	T37, T05, T06, T16	1.5	Service	1
T39	PortfolioServlet + portfolio.jsp	Dev 4	P0	T38, T36	2	Portfolio page	1
T40	HistoryServlet + history.jsp	Dev 4	P0	T06	1.5	History	2
T41	DashboardServlet + dashboard.jsp	Dev 4	P0	T38	1.5	Dashboard	2
T42	error.jsp; verify 403/404/500 mapping	Dev 4	P0	T36	0.5	Error pages	2
T43	UI consistency pass across all JSPs	Dev 4	P0	all UI	2	Polished UI	3
T44	PNL/BUY/SELL execution, test report	Dev 4	P0	T39, T40	2	Test report	3
T45	Docs: testing, screenshots, limitations, future scope	Dev 4	P0	-	2	Report parts	3
T46	Clean-machine deploy test	Dev 4	P0	T15	1	Sign-off	3

Approximate load: Dev 1 ≈ 20 h, Dev 2 ≈ 19 h, Dev 3 ≈ 20 h, Dev 4 ≈ 16 h. Dev 4 absorbs overflow from Dev 1 (T11 test cases) if Day 2 slips.

26. Task Dependency Graph



Preparation (before the examiner arrives): `reset.sql` loaded, Tomcat running, simulator OFF, fault toggle OFF, MySQL Workbench open with these queries ready: `SELECT * FROM users;`, `SELECT * FROM holdings;`, `SELECT * FROM trades ORDER BY id DESC;`. Two browser windows (admin, user). Backup: `mysqldump` file and screenshots.

If the demo goes wrong: reset DB, move on, and never apologise for more than one sentence. Rehearse twice with a timer. Everything above uses features already in the MUST list except the simulator and concurrency demo (P1). Cut those two if they do not exist.

#	Section	Content (concrete)	Owner
1	Title	Project name, team, guide, institute, date	Dev 1
2	Problem statement	Section 2 of this PRD, shortened	Dev 2
3	Objectives	Section 3	Dev 2
4	Proposed solution	Section 1 + architecture overview	Dev 1
5	Scope	Section 6/7, including MUST/CUT	Dev 2
6	User roles	Section 5 table	Dev 2

7	Functional requirements	Section 8 table	Dev 2
8	Non-functional requirements	Offline, UTF-8, response time, security list	Dev 2
9	System architecture	Section 15 diagram + responsibilities table	Dev 2 (Dev 1 reviews)
10	Use case diagram	Actors: Guest, User, Admin. Guest: Register, Login. User: View Market, Search, View Stock, Buy, Sell, View Portfolio, View History, Edit Profile, Logout. Admin: Block/Unblock User, Manage Stocks, Update Price, View Trades, View Stats. Buy/Sell «include» "Validate Balance/Holdings". Draw in draw.io.	Dev 2
11	ER diagram	Section 14; crow's-foot notation, 4 entities	Dev 3
12	Database schema	schema.sql + table summary	Dev 3
13	Class diagram	Packages: model, dao, service, servlet, exception. Show: OrderExecutor < .. AbstractOrderExecutor < -- BuyOrderExecutor/SellOrderExecutor; TradingService o-- OrderExecutor; BaseServlet < -- *Servlet; AppException < .. 7 subclasses; PriceGenerator < .. RandomWalkPriceGenerator; services use DAOs. Do not draw all 60 classes; draw the trading core and one servlet chain.	Dev 2
14	Sequence diagrams	(a) Login; (b) Buy (below); (c) Admin price update	Dev 2
15	Technology stack	Section 30	Dev 3
16	Java concepts used	Section 22 table	Dev 3
17	Testing	Section 23 results (Pass/Fail), bugs found and fixed	Dev 4
18	Screenshots	Every screen + error states + rollback proof (DB before/after)	Dev 4
19	Limitations	Mock data; no CSRF; no HTTPS; single-node; no pagination; market orders only; Western number grouping	Dev 4
20	Future scope	Limit orders, price history charts, watchlist, REST API, Spring migration, connection pool. Keep it short and true.	Dev 4

Buy sequence diagram (steps to draw): 1. Browser → AuthFilter: POST /app/buy; filter checks session and role. 2. AuthFilter → BuyServlet: chain continues. 3. BuyServlet → TradingService.buy(userId, stockId, qty) (userId from the session, never from a form field). 4. TradingService → DBConnection.getConnection(); setAutoCommit(false). 5. TradingService → BuyOrderExecutor.execute(conn, ...). 6. BuyOrderExecutor → UserDAO.findByIdForUpdate; → StockDAO.findById; → UserDAO.adjustCash; → HoldingDAO.findForUpdate/insert/update; → TradeDAO.insert. 7. alt success: TradingService → conn.commit(); BuyServlet → redirect /app/portfolio. 8. alt exception: TradingService → conn.rollback(); exception propagates; BuyServlet → flash error → redirect /app/stock?id=.

Security detail worth mentioning in the report: **userId comes from the session, never from a request parameter.** Otherwise anyone can trade for someone else by editing a hidden field.

Write the documents as you build, not on Day 3 night. Screenshots taken at the end of Day 2 are good backups.

29. Risks & Mitigations

#	Risk	Likelihood	Impact	Mitigation
R1	Tomcat 9 vs 10 mix-up (javax vs jakarta): servlets compile but 404/ClassNotFound	High	High	Standardise on Tomcat 10.1 + Servlet 6 in the first 30 minutes; one pom.xml
R2	JSTL URI error The absolute uri http://java.sun.com/jsp/jstl/core cannot be resolved	High	Medium	Use jakarta.tags.core; JSTL implementation dependency in pom.xml (Tomcat does not ship JSTL)
R3	₹ shown as ?/garbage	High	Low-Medium	utf8mb4 DB, JDBC URL characterEncoding=UTF-8, EncodingFilter, JSP pageEncoding="UTF-8", <meta charset>; test UI-01 on Day 1
R4	MySQL 5.7 ignores CHECK constraints	Medium	Medium	Check SELECT VERSION() on everyone's machine; need ≥ 8.0.16
R5	Merge conflicts in shared files (web.xml, style.css, JSPs)	High	High	File ownership, @WebServlet annotations (no per-servlet web.xml edits), branches, merge only via Leader
R6	Trade code deadlocks or leaves a connection with autoCommit=false	Medium	High	Fixed lock order; finally restores autocommit; try-with-resources; test CON-01/CON-03
R7	Scheduled thread dies silently after one exception	Medium	Low	catch (Throwable) inside the task; log it
R8	Price thread changes values mid-demo	Medium	Medium	Default OFF; explicit toggle; turn OFF before the trade demo
R9	A teammate is blocked or gets sick	Medium	High	Section 26 "write it yourself" rule; all P1 cancelled at checkpoint; Dev 4 has slack
R10	Day-3 integration breaks something	High	High	Twice-daily merges to main; feature freeze at 14:00 Day 3; tagged git tag demo-ready after tests pass
R11	Demo laptop environment differs	Medium	High	Rehearse on the demo laptop; carry WAR + mysqldump + screenshots; no internet dependencies
R12	Scope creep ("let's add charts")	High	High	Section 31 is the answer
R13	Money rounding bugs (double)	Medium	Medium	BigDecimal only; PnLCalculator unit tests
R14	Examiner asks about security gaps	Medium	Low	Documented limitations (Section 20), honest answer
R15	MySQL connector leak warnings on Tomcat redeploy	High	Low	Ignore; restart Tomcat on redeploy if odd

30. Final Build Specification

Final feature list

Register, login, logout, session timeout, role-based access, profile edit and password change; market list + search; stock detail; buy; sell (transactional, locked, validated); portfolio with P&L; history; dashboard; admin: user block/unblock, stock add/edit/deactivate, manual price update, trade monitor, stats; rollback demo toggle. P1: realized P&L display, price simulator thread, concurrency demo, sorting/colours, JUnit.

Final pages (14 JSP + 2 fragments)

login, register, dashboard, market, stock-detail, portfolio, history, profile, error; admin/dashboard, admin/users, admin/stocks, admin/prices, admin/trades; fragments/header.jspf, fragments/footer.jspf.

Final database tables

users, stocks, holdings, trades (Section 13; schema.sql and seed.sql supplied).

Final Java classes (≈ 60)

- **model (11):** User, Stock, Holding, Trade, Role, UserStatus, TradeType, AuthUser, PortfolioRow, PortfolioSummary, PlatformStats
- **dao (5):** UserDao, StockDao, HoldingDao, TradeDao, StatsDao
- **service (6 + 4):** UserService, StockService, TradingService, PortfolioService, PnLCalculator, AdminService; OrderExecutor, AbstractOrderExecutor, BuyOrderExecutor, SellOrderExecutor
- **simulator (3, P1):** PriceGenerator, RandomWalkPriceGenerator, PriceSimulator
- **servlet (17):** BaseServlet + the 16 below
- **filter (2):** EncodingFilter, AuthFilter
- **listener (1):** ApplicationContextListener
- **exception (9):** AppException, ValidationException, AuthenticationException, InsufficientBalanceException, InsufficientHoldingsException, InvalidOrderException, StockNotFoundException, UserNotFoundException, DataAccessException
- **util (6):** DBConnection, PasswordUtil, SessionUtil, Validator, FaultInjector, AppConfig
- **demo (1, P1):** ConcurrentTradeDemo

Final servlets (16)

LoginServlet, RegisterServlet, LogoutServlet, DashboardServlet, MarketServlet, StockDetailServlet, BuyServlet, SellServlet, PortfolioServlet, HistoryServlet, ProfileServlet, AdminDashboardServlet, AdminUsersServlet, AdminStocksServlet, AdminPricesServlet, AdminTradesServlet

Final dependencies (pom.xml)

Artifact	Version	Scope
jakarta.servlet:jakarta.servlet-api	6.0.0	provided
jakarta.servlet.jsp.jstl:jakarta.servlet.jsp.jstl-api	3.0.0	compile
org.glassfish.web:jakarta.servlet.jsp.jstl (implementation)	3.0.1	compile
com.mysql:mysql-connector-j	8.4.0	compile
org.junit.jupiter:junit-jupiter	5.10.x	test (P1)

Packaging war; maven.compiler.release 17; maven-war-plugin 3.4.0 with failOnMissingWebXml=false (you will have a web.xml anyway). That is every dependency. The JSTL artifact pair (jakarta.servlet.jsp.jstl-api 3.0.0 + org.glassfish.web:jakarta.servlet.jsp.jstl 3.0.1) and the jakarta.tags.core URI were checked against current documentation and Maven Central listings. Keep **both** JSTL jars: a GitHub issue shows people hitting ClassNotFoundException: jakarta.servlet.jsp.jstl.core.ConditionalTagSupport when the API jar is missing. Servlet 6.0.0 and Connector/J 8.4.0 versions were not re-checked; confirm on Maven Central when you set up.

web.xml must use the Servlet 6.0 header, otherwise Tomcat can fall back to an older mode in which JSTL/EL misbehave:

```
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="https://jakarta.ee/xml/ns/jakartaee https://jakarta.ee/xml/ns/jakartaee/web-app_6_0.xsd"
  version="6.0">
  ...
</web-app>
```

Environment: JDK 17, Tomcat 10.1.x, MySQL 8.0.x (≥ 8.0.16), Maven 3.9.x, Git.

Final team assignments, schedule, demo

Sections 24, 25, 27.

31. DO NOT BUILD

Feature	Reason
Real money, payment gateway, bank integration	Out of project definition; legal risk; zero marks
Real broker API, live NSE/BSE data, external market-data API	Needs internet, keys, rate limits; one failure kills the demo
orders table, limit/stop/bracket orders, order book, matching engine	Weeks of work; market orders demonstrate the same transaction logic
Intraday margin, short selling, options, futures	Different accounting model
Brokerage fees, taxes, STT, charges	Breaks the clean P&L invariant for no learning gain
Technical analysis, indicators, candlestick charts, price history	Charting is a frontend project of its own
Algorithmic trading, AI prediction, sentiment analysis	Fantasy
KYC, PAN, Aadhaar, OTP, email verification, forgot password	Needs external services; irrelevant to the learning goals
WebSockets / live push	Refresh the page
Microservices, Docker/Kubernetes, cloud deployment, CI/CD	Nobody will mark you on it; each costs hours
Spring Boot / Hibernate / JPA	Hides the Servlet and JDBC concepts you are being graded on
Mobile app, REST/SPA frontend, React/Angular	Doubles the frontend work
Connection pool (HikariCP/DBCP)	Not needed for demo load; mention in future scope
Separate admin login page	Same login, role redirect
Buy/sell confirmation page	An extra servlet and JSP for nothing
User deletion, admin balance reset	FK complexity; breaks the P&L invariant
Pagination, sorting beyond P1, export to CSV/PDF	Cut
Watchlist, alerts, notifications, news	Cut
Multi-currency, i18n	Cut
CSRF tokens, rate limiting, 2FA	P2 at best; document as limitations
Admin-defined roles/permissions	Two roles are enough
Custom JS framework, animations, dark mode	CSS only

32. Definition of Done

The project is done when **all** of the following are true:

1. `mvn clean package` succeeds from a fresh clone, and the WAR deploys on a clean Tomcat 10.1 with a freshly seeded MySQL.
2. Every P0 test in Section 23 passes, or the failure is documented with a reason.
3. The Day-2 flow works end to end: register → buy → portfolio → sell → history → admin sees the trade.
4. Rollback demo (TX-01/TX-02) shows no change in `users`, `holdings`, `trades`.
5. Overspend is impossible: BUY-03 passes, and (if built) CON-01 passes.
6. No SQL built by string concatenation (search the codebase for "+" inside SQL strings).
7. No scriptlets (`<% %>`) in JSPs, and no business logic in servlets beyond calling a service.
8. No `double/float` used for money (search for it).
9. All user-supplied output goes through `<c:out>`.
10. ₹ renders correctly on every page.
11. All P0 documentation sections written, screenshots included.
12. Demo performed twice, start to finish, within 10 minutes, on the demo laptop, with internet disabled.
13. `main` tagged `demo-ready`; `mysqldump` and WAR backed up on a USB drive and cloud storage.